The Models of Pathfinder

Architecture, Training, and Algorithmics of a

Serverless Music-Journey Engine

LENSING · spotify-pathfinder
`spotify-predict-engagement` → `app` (Cloudflare Worker + WASM)

June 2026

Abstract

Pathfinder renders a "musical journey" between two Spotify tracks: an A* search threads a path through a 23,529-track map of listening taste, drawn in 3D. The system is unusual in that it runs *entirely* on a single Cloudflare Worker — no Python origin, no vector database at request time — by compiling all numeric work to WebAssembly and baking a static snapshot of every model's *output* into the deployed artifact. This report documents the seven models and algorithms behind it: a song autoencoder that defines the taste map, a gradient-boosted classifier that scores track "fit", a distilled cold-start projector, a satisfaction-weighted transition model, a symmetric k-NN graph, the A*+densify pathfinder, and a power-iteration PCA for the 3D view. For each we give structure, the training recipe, and the design rationale — grounding every claim in the `lensing` training project and clearly flagging what is documented versus inferred. We include error surfaces from real parameter sweeps: a freshly-run hyperparameter scan of the rotation classifier on its true champion dataset, and response surfaces of the A* cost weights over the real corpus.

1 System overview: the lensing platform

The models were produced by **lensing**, a prediction-lab framework that "shapes itself around your dataset the way mass bends light": point it at a Qdrant corpus, declare a target and fields in `domain.toml`, and it bakes datasets, trains a registry of candidate predictors, and promotes the best to a named export. The Pathfinder `app` is the serverless *inference* instance of one such lensing problem (`spotify-predict-engagement`).

The defining architectural split is **train offline, infer at the edge** (Figure 1). Every learned model runs once, offline, over the corpus; its outputs — 64-d latent vectors, per-track fit scores, transition tables, the k-NN graph — are written to binary snapshot files (`corpus.bin`, `transitions.json`, `projector.bin`). At request time the Rust/WASM core reads those frozen outputs and does *pure arithmetic*: search, densify, PCA. An in-corpus → in-corpus route touches *zero* live model inference. Only off-map (cold-start) tracks invoke a model at request time — a text embedding plus the projector — because the geometry for arbitrary tracks cannot be precomputed.

Why this shape. The Workers Free plan caps each request at ∼10 ms of CPU. Live gradient-boosting or neural inference over 23k tracks cannot fit; precomputing every model output and shipping it as a memory-mapped snapshot does. The cost is staleness: refreshing the corpus means re-running the offline build and redeploying.

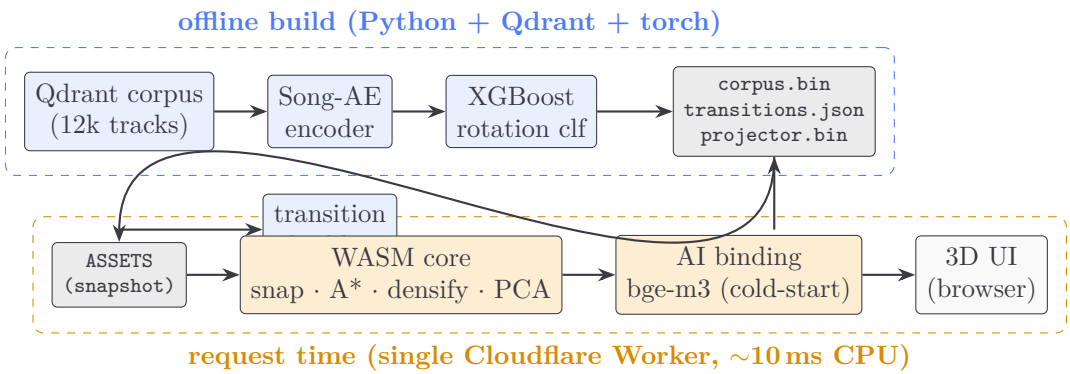

Figure 1: The train-offline / infer-at-the-edge split. Learned models (blue) run once; the Worker (orange) reads their frozen outputs and does only arithmetic, except for the cold-start embedding.

Table 1 summarizes the seven components. The rest of the paper takes them in turn.

Table 1: The seven models / algorithms. "Runs" is when inference happens.

Component	Type	In → Out	Runs
Song-AE	autoencoder (MLP)	539 → 64	offline (defines the map)
XGBoost rotation clf	gradient-boosted trees	115 → 1	offline (bakes `fit`)
Cold-start projector	MLP (distilled)	1179 → 64	request (off-map only)
Transition model	weighted bigram + back-off	meta → affinity	request (in WASM)
k-NN graph	symmetric cosine, k=20	—	offline (baked edges)
A*+densify	heuristic graph search	graph → path	request (in WASM)
PCA	power iteration + deflation	64 → 3	request (per route)

2 The taste map: the Song autoencoder

Role. The Song-AE's 64-dimensional latent *is* the coordinate system everything else lives in. Cosine distance between two latents is the system's notion of musical distance; the k-NN graph, the A* heuristic, and the 3D PCA all operate on these vectors. They are L2-normalized, so cos similarity reduces to a dot product.

Structure (`pipeline/build_song_ae.py`). A symmetric undercomplete autoencoder:

$$\text{enc}: \mathbb{R}^{539} \xrightarrow{W_1} \mathbb{R}^{256} \xrightarrow{\text{ReLU}} \xrightarrow{W_2} \mathbb{R}^{64}, \qquad \text{dec}: \mathbb{R}^{64} \to \mathbb{R}^{256} \xrightarrow{\text{ReLU}} \mathbb{R}^{539}.$$

The 539-d input concatenates five blocks of heterogeneous evidence about a track (Figure 2, left): a 384-d multilingual text embedding (`paraphrase-multilingual-MiniLM-L12-v2`) of a natural-language "content document"; 10 z-scored numerics (popularity, followers and duration via $\log 1p$, release year, ...); 22 acoustic dims (11 ReccoBeats audio features + 11 missing-flags, §5); a 120-way genre multi-hot; and a 3-way album-type one-hot.

Training. Block-wise mean-squared reconstruction, summed with equal weight per modality so a large block (text) cannot dominate a small one (album):

$$\mathcal{L} = \sum_{b \in \text{blocks}} \left\| \hat{x}_{[b]} - x_{[b]} \right\|_2^2.$$

Adam (lr $= 10^{-3}$, weight decay 10^{-5}), batch 256, 200 epochs, seed 42, all 12,256 tracks (no held-out split — the goal is a faithful corpus embedding, not generalization to unseen tracks). The encoder weights are frozen into `song_ae.pt`; the deployed corpus latents are its outputs.

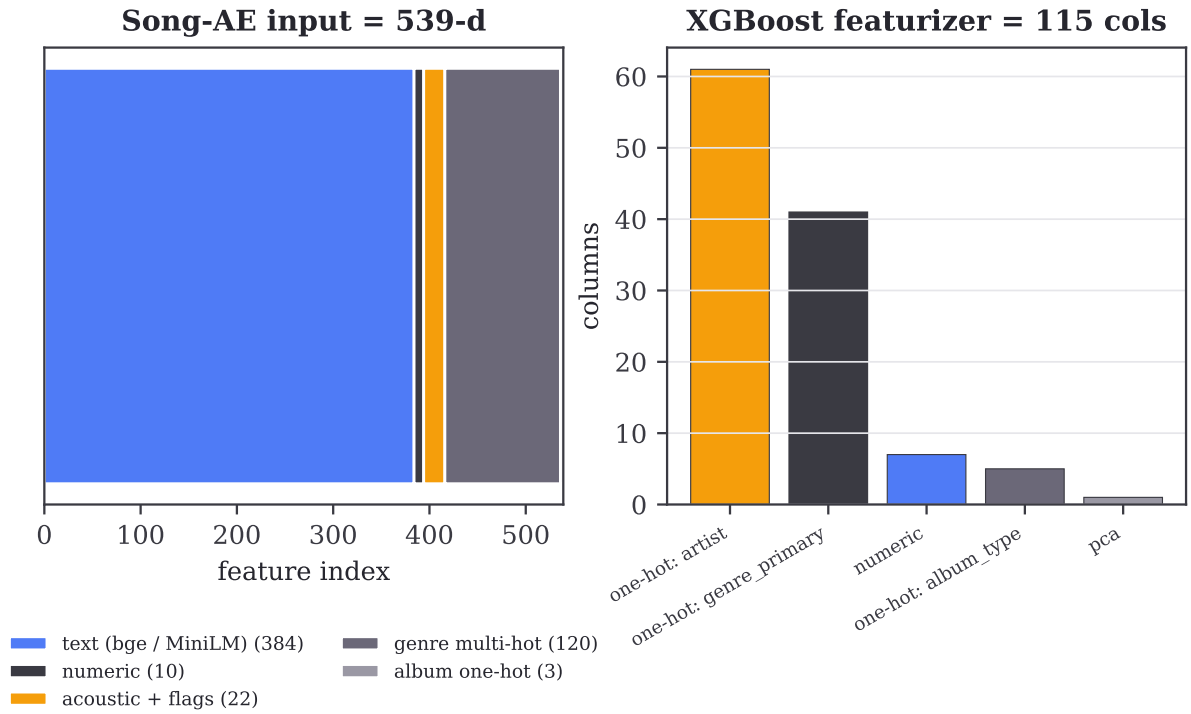

Figure 2: Left: the 539-d Song-AE input as five concatenated modality blocks. Right: the *separate* 115-column featurizer the XGBoost classifier actually uses — dominated by identity one-hots (artist, genre), not dense vectors (§4).

Documented. Architecture, loss, and optimizer are read directly from `pipeline/build_song_ae.py`. *Not* documented: why 64 latent dims specifically, or a study of deeper encoders — the latent width is a fixed constant across the system.

3 Engagement / fit: the XGBoost rotation classifier

What it predicts. Each track gets a scalar `fit` $\in [0, 1]$ used by A* to prefer "sticky" tracks. `fit` is *not* a regression of play-count; it is the calibrated probability of **rotation**, the binary target `rotation = (play_count` $\geq$ `2)` — "did the listener come back to this track?" (`domain.toml`). The model is an XGBoost binary classifier (`objective=binary:logistic`); its positive-class probability is exported to ONNX and, at snapshot-build time, scored over the whole corpus, min–max normalized, and written as the `fit` column of `corpus.bin`.

Why rotation, not engagement. The project originally targeted `engagement = play_count` $\times$ `completion_ratio` ($\log 1p$). It pivoted to rotation because the binary taste-fit signal is more learnable (AUC ~0.73–0.77) and spreads naturally into a $[0, 1]$ fit weight (`PROJECT-FACTS.md`). The shipped champion scores **AUC 0.7429**, logloss 0.5883, accuracy 0.6846 on a 2,451-track held-out split.

Features. A *frozen* 115-column featurizer (Figure 2, right), distinct from the Song-AE's 539-d input: 1 PCA floor component of the latent, a handful of numerics, and ~107 one-hot columns for `artist`, `genre_primary`, and `album_type` (each with an `__other__` catch-all). Hyperparameters (`predictors/xgboost-classifier/train.py`): 500 trees, `max_depth` 6, `learning_rate` 0.05, `subsample`/`colsample_bytree` 0.8, `reg_lambda` 1.0, seed 42.

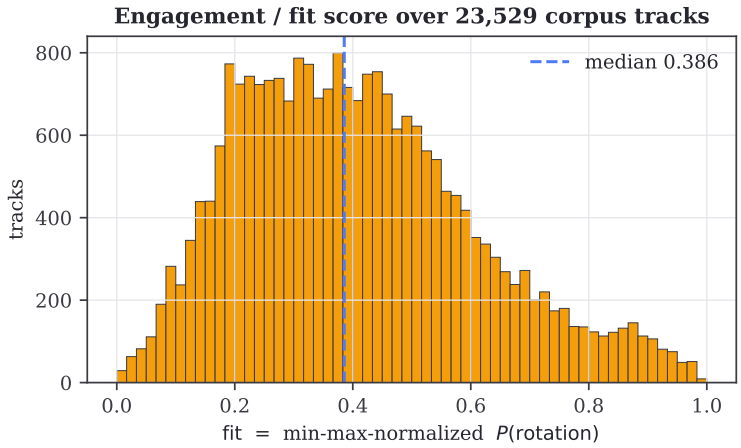

Figure 3: The `fit` column actually shipped in `corpus.bin`: the min–max-normalized rotation probability for all 23,529 corpus tracks. A* pays $W_{\mathrm{FIT}}(1 - \texttt{fit})$ per hop (§8).

3.1 An error hypersurface: the deferred hyperparameter scan

The champion's hyperparameters are sensible defaults, *not* the result of a grid search. `PROJECT-FACTS.md` explicitly names "a depth / n_estimators / learning_rate scan" as "the next real lever for AUC" and *defers* it. We ran that deferred scan for this report, directly on the real champion dataset `ds-20260609-141622-p1-s42` and frozen split, using the exact reference training path (`XGBClassifier`, AUC on positive-class probability). Our harness reproduces the published 0.7429 to four decimals before sweeping, so the surface is trustworthy.

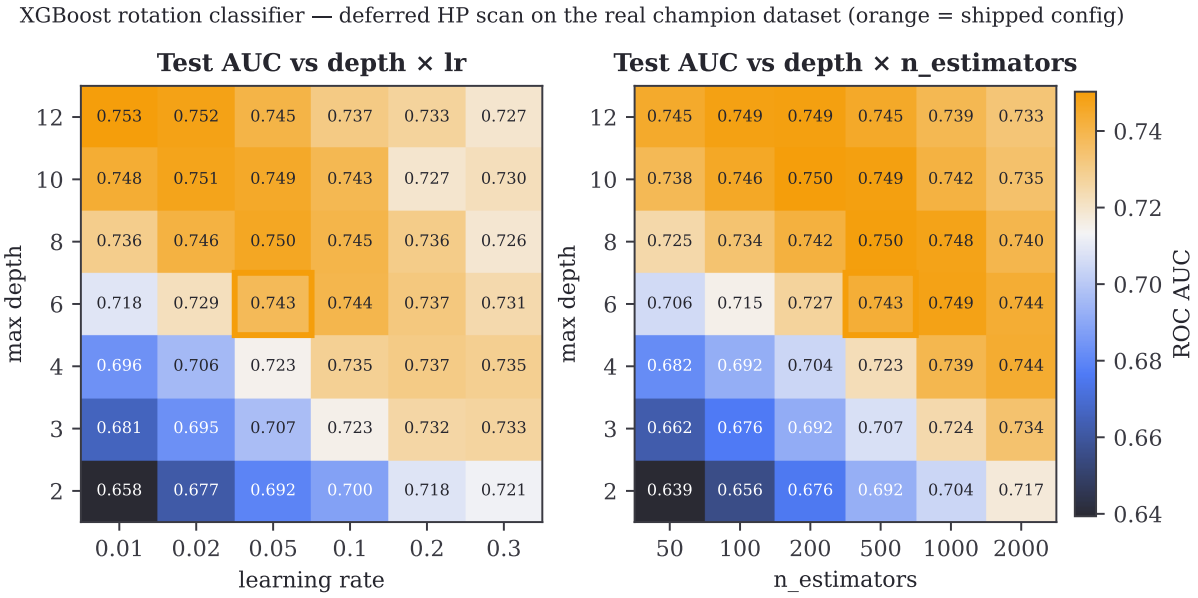

Figure 4: Test-AUC error surfaces from the deferred scan (real data, computed for this report). Left: `max_depth` × `learning_rate`; right: `max_depth` × `n_estimators`. The orange cell is the shipped config. The classic boosting ridge is visible (low lr wants more depth/trees). Best cell: AUC **0.7534** at depth 12, lr 0.01 — only +0.011 over the champion, *inside* the measured $2\sigma = 0.0126$ noise band (§4). The scan was reasonably deferred: no cell is a significant win.

Provenance. The numbers in Figure 4 are genuine — 84 XGBoost fits on the real frozen split (`docs/sweeps/run_xgb_sweep.py`). They are *new* work, not a recovered training log, because no such grid was ever stored. The takeaway is honest: tuning moves AUC less than seed noise.

4 Why identity beats dense vectors (the central lesson)

The most strongly documented finding of the whole project is a negative one about features. Across two targets and six confirmations, **artist/genre identity one-hots exploited by trees beat every dense representation**: the co-listening embedding, the 384-d content-text embedding, the 11 acoustic features, and — notably — the Song-AE's own 64-d latent all *hurt or diluted* the score. "The taste-fit signal is predicted by artist/genre IDENTITY . . . not by descriptive attributes nor by distance/margin models" (`PROJECT-FACTS.md`).

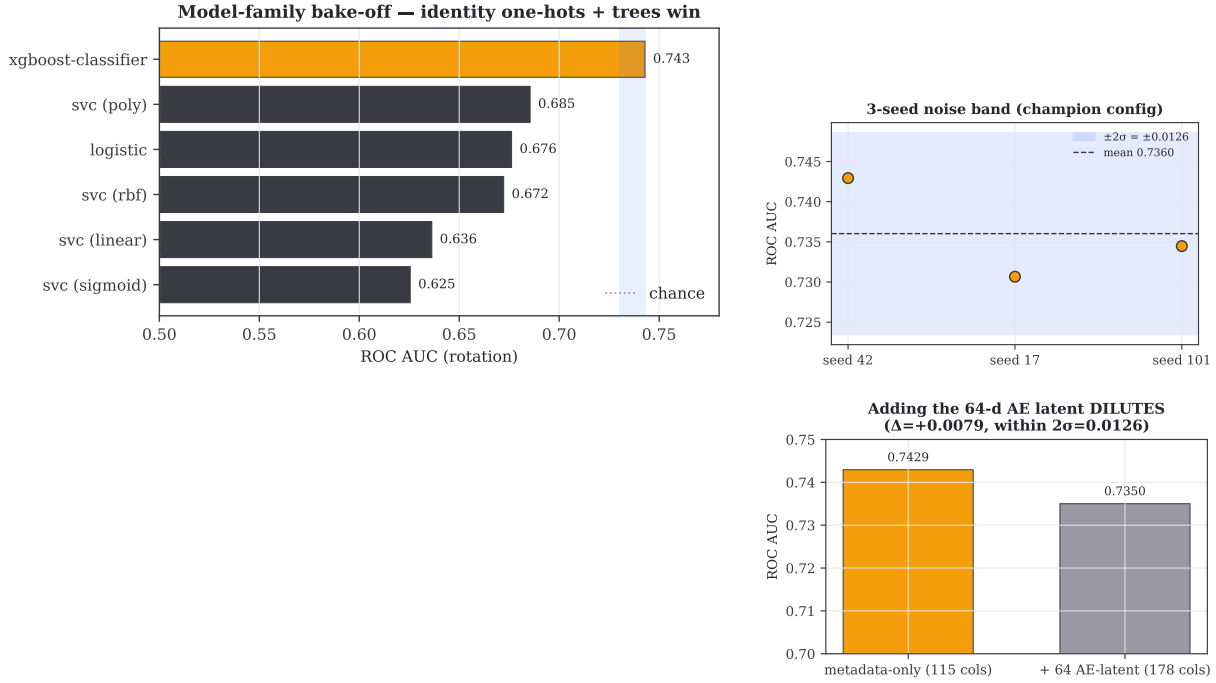

Figure 5: Real logged runs on the rotation champion dataset. **Left:** family bake-off — the tree beats every dense distance/margin model (SVC kernels, logistic) by 4.6–9.3 noise bands. **Top-right:** the 3-seed noise band that calibrates significance ($\sigma_{\text{AUC}} = 0.0063$). **Bottom-right:** adding the 64-d AE latent as features *dilutes* AUC by 0.0079.

This is why the Song-AE latent and the fit-prediction featurizer are *deliberately separate* (Figure 2): the latent is excellent as a *geometry* for the map (it places similar tracks near each other) yet a poor *feature* for predicting rotation, where raw identity carries the signal. A meta-lesson the project states plainly: "features usually beat architecture; scan dataset-level axes before deep hyperparameter scans" — consistent with the flat surface in Figure 4.

Pitfalls documented. The project records concrete traps: SVR with an ε-tube on a {0,1} target posts a deceptively low MAE while being worse-than-mean on R^2; MAE-tuned blends collapse their weight onto that artifact ($w \to 0$). These motivated switching to AUC/logloss as the honest classification metrics.

5 Cold-start: projecting an off-map track

When a requested track is not in the corpus, it must be placed in the 64-d latent space at request time. Two instances solve this differently:

- **Reference** (`pipeline/predict_spotify_url.py`): run the *actual* frozen Song-AE encoder, then

the dataset's frozen PCA basis. Needs torch + sentence-transformers.

- **App** (`worker-core/src/project.rs`, built by `tools/build_snapshot.py`): a Worker cannot run torch, so the app ships a **distilled projector** — a small MLP $\mathbb{R}^{1179} \to 256 \to 64$ trained to *reproduce* the corpus Song-AE latents from features obtainable live (a 1024-d `bge-m3` embedding from the Workers AI binding, plus the same numeric/acoustic/genre/album blocks). Loss is cosine distance to the L2-normalized target latent; Adam 10^{-3}, batch 512, 120 epochs; validated by self-recall@1.

Both land the track in the same space, after which it *snaps* to the nearest corpus anchor and routing proceeds. The projector is a lossy *translator* from "features available in the Worker" to "the Song-AE representation", which is why the app then trusts the nearest real anchor over the raw projected point.

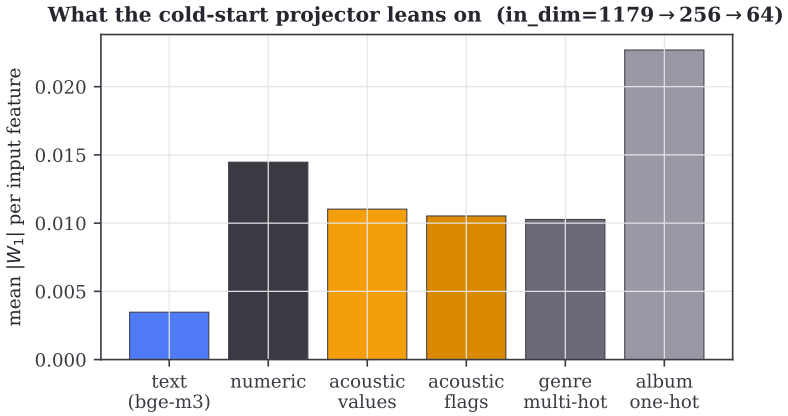

Figure 6: Mean $|W_1|$ per input feature in the deployed projector (`projector.bin`), by input block. The text embedding and numerics carry most of the first-layer weight; acoustic *missing-flags* carry real signal too (below).

The missing-flag block. Acoustic features are often absent. Imputing the mean is fine, but because features are z-scored, an imputed value becomes exactly 0 — *indistinguishable from a genuinely average track*. Each acoustic feature therefore contributes *two* inputs: a value and a 0/1 "was-this-real" flag (laid out as two contiguous blocks, matching the trained weight order):

Table 2: Mean-imputation made honest. Rows 2–4 all yield value 0.0 for different reasons; the flag disambiguates "placeholder" from "genuinely average".

feature	raw	stats (μ, σ)	value slot $= (x - \mu)/\sigma$	flag
danceability	0.8	$(0.6, 0.2)$	$+1.0$	0
energy	— (missing)	$(0.65, 0.15)$	0.0 *(imputed)*	**1**
valence	0.5	$(0.5, 0.2)$	0.0 *(real)*	0
tempo	120	$(120, 30)$	0.0 *(real)*	0

6 The transition model: what naturally follows what

A* does not move on geometry alone; it also prefers transitions that real listening sessions actually make. The transition model (`pipeline/corpus/transitions.py`, ported to `worker-core/src/transit.rs`) is built from a sessionized play log. Consecutive non-skip plays (≥ 30 s) form a **track bigram**, each edge weighted by the destination's *satisfaction* (e.g. `trackdone`=1.0, `fwdbtn`=0.25). Bigrams are aggregated up to artist→artist and genre→genre conditionals.

Affinity with back-off. Track-level data is sparse — most tracks have only a handful of recorded successors — so the next-track affinity blends the track-specific bigram with a coarser artist/genre estimate, trusting the bigram only as far as its evidence goes:

$$\mathrm{aff}(u, v) = \tau\, P(v \mid u) + (1 - \tau) \underbrace{\big[0.6\, P_{\mathrm{art}} + 0.4\, P_{\mathrm{gen}}\big]}_{\text{back-off prior}}, \qquad \tau = \frac{n_u}{n_u + K}, \quad K = 8.$$

The *support* n_u is what drives τ: it is the total number of times track u was followed by a kept (non-skip) track anywhere in the log — the amount of evidence behind $P(v \mid u)$. The trust weight $\tau \in [0, 1)$ is just that evidence divided by itself plus a constant K, where K acts as a *pseudo-count* — as if we had pre-seen K plays that vote for the back-off prior. In words, τ = "real observations / (real + K imaginary) observations":

$n_u = 0 \;\Rightarrow\; \tau = 0$ (pure back-off), $\quad n_u = K = 8 \;\Rightarrow\; \tau = 0.5$ (half each), $\quad n_u = 40 \;\Rightarrow\; \tau \approx 0.83$ (mostly bigram)

So a track seen only a few times leans on its artist/genre statistics, while a heavily-observed track trusts its own successors; $K = 8$ is the crossover support at which the two carry equal weight (Figure 8).

Context: what a "lift" is. Listening also has a rhythm — some genres belong to the evening, some to a shuffle session — and the model captures this with *lifts*. A lift is just a multiplier that says how much more (or less) a genre is played in a given context than it is overall:

$$\mathrm{lift}(g \mid c) \;=\; \frac{P(g \mid c)}{P(g)} \;=\; \frac{\text{rate of genre } g \text{ within context } c}{\text{rate of } g \text{ across the whole log}}.$$

So lift = 1 is neutral (the context makes no difference), lift > 1 *boosts* the genre (it is over-represented there), and lift < 1 *suppresses* it. If jazz plays twice as often in the evening as it does on average, its evening lift is 2.

A candidate track v collects the product of three such lifts — its genre under the current time-of-day, its genre under shuffle-vs-linear, and (for sufficiently popular tracks) the track itself under the time-of-day:

$$\mathrm{ctx}(v) = \mathrm{lift}_{\mathrm{tod}}(g_v) \cdot \mathrm{lift}_{\mathrm{shuf}}(g_v) \cdot \mathrm{lift}_{\mathrm{tod}}(v).$$

Two guards keep these honest: each lift is *shrunk toward* 1 in proportion to how little data supports it (a genre seen only a few times in a context stays near neutral instead of swinging to an extreme), and the product is *clipped to* $[0.2, 5]$ so no single context can dominate. This $\mathrm{ctx}(v)$ is exactly what `context_fit` returns; like the affinity above, it is min–max normalized across the expansion's candidates before it enters the A* cost as the W_{CTX} term — only the *relative* context preference among the current options matters.

Lifts in action. Figure 7 (left) shows real genre lifts from this listener's log (track counts in parentheses). The patterns are sharp: *alternative metal* is a morning sound (1.67× in the morning, only 0.75× at night), *dance pop* is its mirror image (1.40× at night), and *indie rock* and *art pop* both peak in the afternoon (1.29×). The deliberate/shuffle axis splits them too: metal and indie lift under shuffle, while dance pop and big beat lift under linear, in-sequence play. A* reads these multipliers through `context_fit` — an afternoon context quietly makes art pop cheaper to route through.

The same study on a different feature. Running the *identical* lift definition on **release era** (Figure 7, right) shows the daily rhythm is largely a *genre* phenomenon: era lifts hug 1 and the panel is nearly colorless. The only mild tendencies are that 2020s tracks skew toward mornings (1.15×) and toward linear, deliberate play (shuffle just 0.86×), while the oldest tracks tick up at night (1.1×). The lesson generalizes: context governs *what kind* of music far more than *when* it was made — not every feature earns a place in the W_{CTX} term.

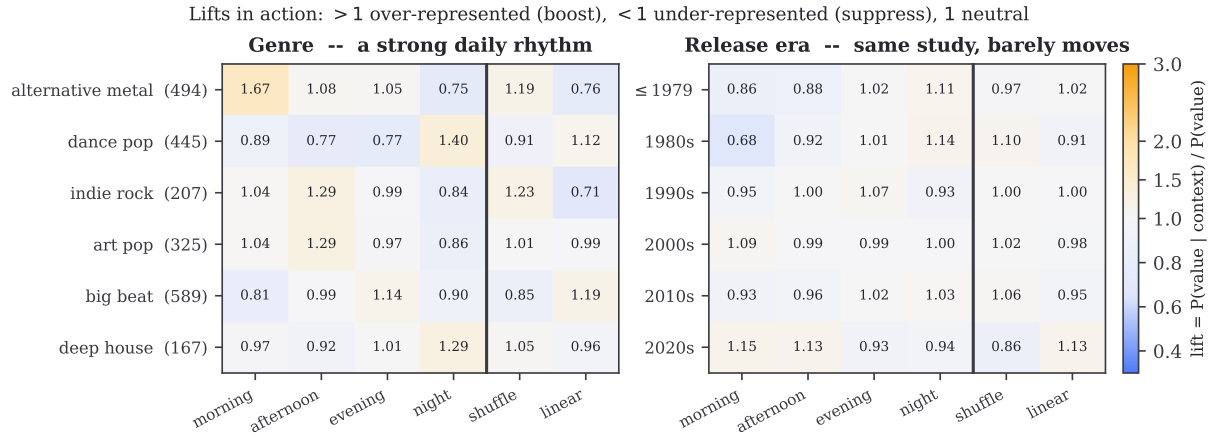

Figure 7: Real lifts from the listening log, computed with one definition for both panels (track counts in parentheses). Orange = over-represented in that context (boost), blue = under-represented (suppress), near-white = neutral (≈ 1); the rule separates time-of-day from shuffle state. **Left:** genre shows a strong daily rhythm. **Right:** release era, run through the same pipeline, barely departs from neutral — the rhythm is about genre, not era.

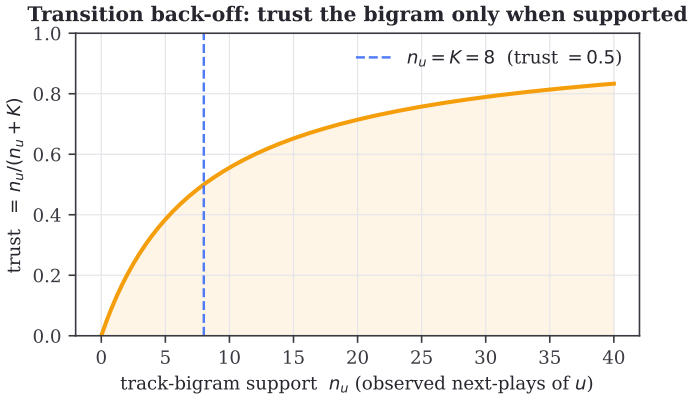

Figure 8: Back-off trust $\tau = n_u/(n_u + 8)$, where the support n_u is how many recorded successors track u has. Below $K = 8$ observed next-plays the model leans on the artist/genre back-off; above it, it trusts the track bigram. Both `affinity` and `context_fit` are min–max normalized across each expansion's candidates before entering the A* cost, so only *relative* preference matters locally.

7 The k-nearest-neighbor graph

A* needs a sparse graph, not all-pairs distances. Offline, each track is connected to its $k = 20$ nearest neighbors by cosine distance; edges are made **symmetric** (if A is near B, B links A) and sorted ascending. The graph (indices + distances) is baked into `corpus.bin`, so at request time edge distances are a table lookup — no vector math for graph moves. Symmetry also widens the candidate pool that `densify` (§8) draws from.

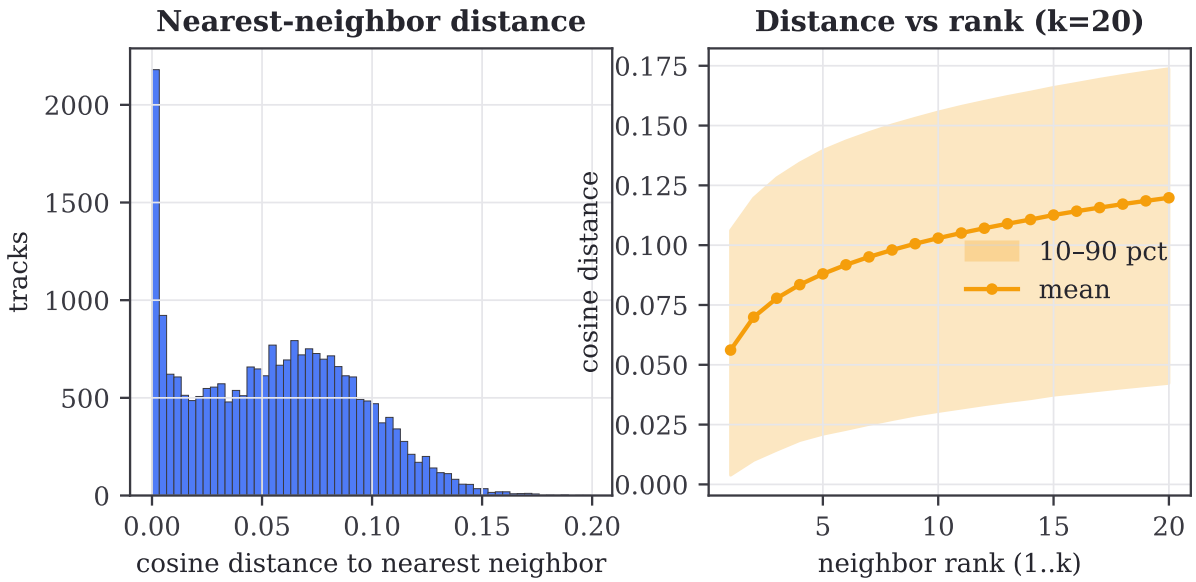

Figure 9: The real neighbor graph in `corpus.bin`. Left: distribution of the nearest-neighbor cosine distance over all 23,529 tracks. Right: mean (and 10–90 percentile band) of cosine distance versus neighbor rank $1 \ldots 20$ — the local manifold density the search traverses.

8 Pathfinding: A* with densify

The search. Pathfinder runs A* on the k-NN graph (`worker-core/src/pathfind.rs`, a faithful port of `pathfinder/search.py`). The heuristic is the cosine distance straight to the goal; the per-hop cost blends five interpretable, weighted terms:

$$\text{step}(u \to v) = \underbrace{W_{\text{D}}\, d(u,v)}_{1.0} + \underbrace{W_{\text{F}}(1 - \texttt{fit}_v)}_{0.5} + \underbrace{W_{\text{V}}\,[g_v \neq g_u]\,\gamma}_{0.5,\ \gamma=0.15} + \underbrace{W_{\text{T}}(1 - \widetilde{\text{aff}})}_{0.6} + \underbrace{W_{\text{C}}(1 - \widetilde{\text{ctx}})}_{0.4},$$

where $\widetilde{(\cdot)}$ are the per-expansion min–max normalized transition and context scores. Hard constraints forbid repeats, > 2 consecutive same-artist tracks, and any artist exceeding $\lceil L/4 \rceil$ of an L-track route. If the found path is shorter than the requested length, `densify` greedily inserts the best-scoring intermediate track into the widest remaining gap.

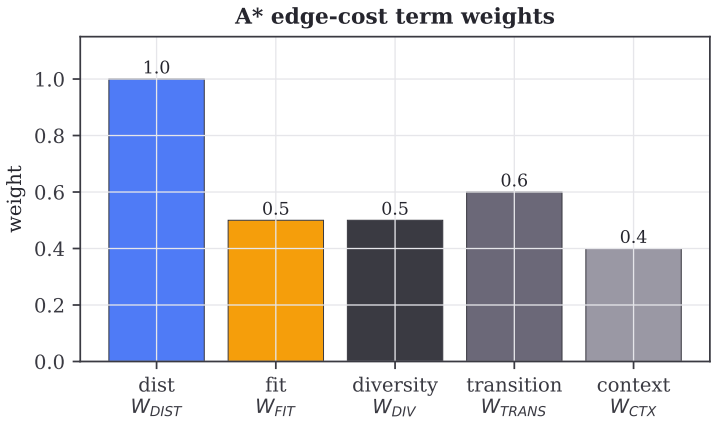

Figure 10: The five A* edge-cost weights (`pathfinder/config.py`). Distance dominates; transition history (0.6) outweighs time-of-day context (0.4); fit and genre-diversity are equal soft terms.

The CPU budget shapes the algorithm. A faithful port is not enough: on the Free plan a far-apart pair can blow the ∼10 ms slice. The WASM core therefore caps the search at `MAX_EXPANSIONS = 45,000` and returns a clean "no path" (404) rather than being killed mid-search (503). This is a deployment constraint leaking into algorithm design, not a modeling choice.

8.1 Response surfaces of the cost weights

The five weights are hand-set constants. To show how the *found path* responds to them, we re-ran the actual search (numpy port over the real `corpus.bin` / `transitions.json`) on three fixed routes across two weight grids (Figure 11). These are *response surfaces of a deterministic search*, not trained-model error.

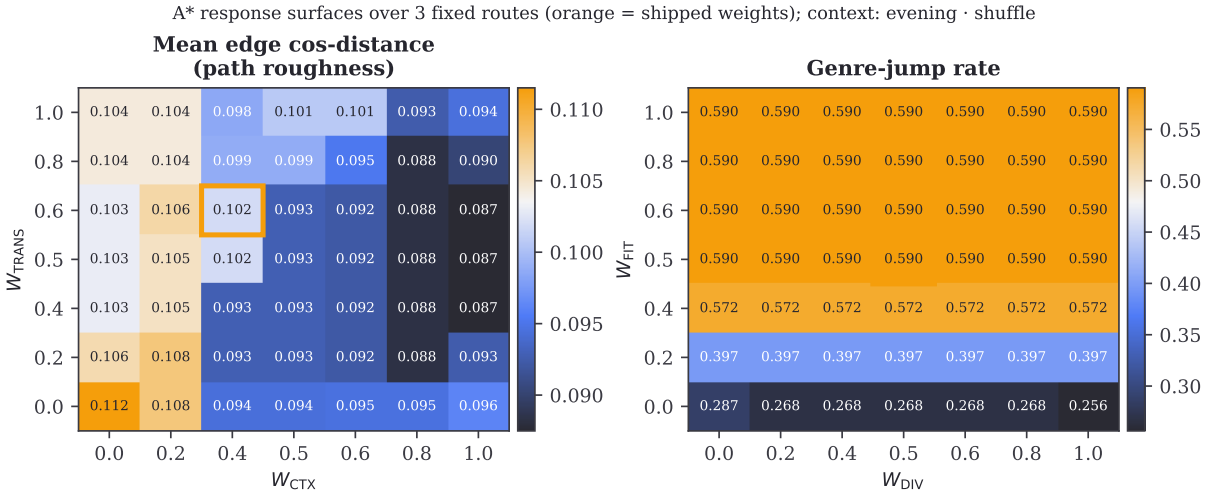

Figure 11: A* response surfaces over the real corpus (orange = shipped weights). **Left:** raising W_{CTX} (and W_{TRANS}) yields measurably *smoother* paths (lower mean edge distance). **Right:** genre-jump rate climbs with W_{FIT} (chasing high-fit tracks pulls across genres) but is nearly *flat* in W_{DIV} — the genre penalty $0.15\,W_{\text{DIV}}$ is too small to redirect these routes, confirming it is a gentle soft constraint. Search capped at 15,000 expansions for tractability in Python.

9 The 3D view: PCA by power iteration

For rendering, the path and its neighborhood "cloud" are projected from 64-d to 3-d per request (`worker-core/src/pca.rs`). To avoid a dense SVD dependency in WASM, the top-3 covariance eigenvectors are found by **power iteration with deflation** — cheap, since the covariance is only 64×64 over ∼100 points — then scaled by the 90th-percentile radius to ± 8. Those axes capture most of the local variance (Figure 12).

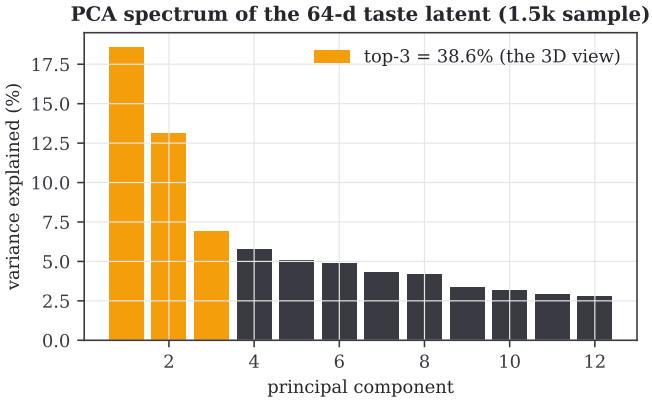

Figure 12: Variance spectrum of the 64-d latent (1,500-track sample); the top-3 axes (orange) used by the 3D view capture a large share, so the projection faithfully summarizes local structure.

10 Validation and parity

Because the request-time core is a Rust rewrite of Python references, correctness is enforced by tests, not assumed. A **parity test** (`worker-core/tests/parity.rs`) checks the Rust A* against `search.py` on corpus routes: identical endpoints/length, all constraints satisfied, Rust cost $\leq 1.6\times$ Python cost $+0.1$, and $\geq 85\%$ node identity on at least one case. The tolerance is deliberate — heuristic search legitimately yields alternate near-optimal paths under float reordering, so exact byte-match is the wrong bar. A second test round-trips the projector binary to guard the `PFP1` format. Significance of *model* differences is judged against the measured noise band $\sigma_{\text{AUC}} = 0.0063$ (Figure 5), not eyeballed.

11 Lessons and rationale

- **Identity > architecture** (documented). One-hot artist/genre features in trees beat every dense embedding and every margin model; the Song-AE latent, useful as *geometry*, dilutes as a *feature*. Scan feature/dataset axes before tuning models.
- **Tuning < noise, here** (this report). The deferred depth/lr/n_est scan (Figure 4) moves AUC by less than the 3-seed band — evidence the deferral was correct and the champion defaults are adequate.
- **Honest metrics** (documented). The pivot to rotation+AUC/logloss followed concrete failures of MAE on a binary target (the ε-tube artifact, blend-weight collapse).
- **Frozen outputs beat live inference** (architecture). Baking model outputs into a snapshot is what lets seven models serve from a 10 ms Worker; the cost is a rebuild-to-refresh cycle.
- **Two latent uses, one space** (architecture). The projector exists only because a Worker cannot run the real encoder; it is a deliberately lossy translator, backed by a snap-to-anchor step.

A Snapshot binary formats

Table 3: `corpus.bin` (`PFC1`) and `projector.bin` (`PFP1`) layouts, little-endian. n=23,529, dim =64, k=20; projector in=1179, h=256, out=64.

`PFC1` corpus.bin		`PFP1` projector.bin	
field	type	field	type
magic / ver / n / dim / k	`u32`×5	magic / ver / in / h / out / text	`u32`×6
ids	`u64`[n]	W_1	`f32`[h·in]
vecs (L2-norm)	`f32`[n· dim]	b_1	`f32`[h]
fit	`f32`[n]	W_2	`f32`[out·h]
nbr_idx	`u32`[n·k]	b_2	`f32`[out]
nbr_dist	`f32`[n·k]	cfg_len	`u32`
meta_len + JSON	`u32`+bytes	cfg (stats, vocab)	JSON

B Constants

A* cost	value	transition / search	value
W_{DIST}	1.0	K (back-off)	8.0
W_{FIT}	0.5	artist / genre weight	0.6 / 0.4
W_{DIV}	0.5	max consecutive artist	2
W_{TRANS}	0.6	artist share divisor	4.0
W_{CTX}	0.4	MAX_EXPANSIONS	45,000
genre-jump γ	0.15	k-NN k	20

Table 4: Cost weights and constants (`pathfinder/config.py`, `transit.rs`).

XGBoost champion	value	data
n_estimators / max_depth	500 / 6	dataset `ds-...-141622-p1-s42`
learning_rate	0.05	115 cols, 9805 train / 2451 test
subsample / colsample	0.8 / 0.8	AUC 0.7429, logloss 0.5883
reg_lambda / seed	1.0 / 42	$\sigma_{\text{AUC}} = 0.0063$ (3 seeds)

Table 5: Rotation classifier (`predictors/xgboost-classifier/train.py`; metrics from `data/runs/`).

C Undocumented / open questions

For honesty, what the source artifacts do *not* establish: (i) why 64 latent dims, or any ablation of Song-AE depth/width; (ii) XGBoost tree-level tuning provenance — the grid in §3.1 was run by us, not recovered; (iii) smoothing/thresholds in the transition-table construction beyond the satisfaction weights and support shrinkage; (iv) provenance of the A* weight values — they are constants with no recorded tuning study (our Figure 11 probes behavior, not optimality); (v) end-to-end request latency and cold-start path behavior on deployed hardware, which `WORKLOG.md` notes is untested.

D Reproducibility

Every figure regenerates from the repository. Sweeps (write `docs/sweeps/*.json`):

```
predictors/.venv/bin/python docs/sweeps/run_xgb_sweep.py
$ANACONDA/bin/python docs/sweeps/run_astar_sweep.py
$ANACONDA/bin/python docs/sweeps/load_runs.py
```

Figures and document:

```
$ANACONDA/bin/python docs/make_figures.py
cd docs && tectonic models.tex
```